

Conteúdo do Relatório

1. Introdução e Mapeamento de Funcionalidades

Neste trabalho, aplicamos os conceitos de estruturas de dados hierárquicas através de dois módulos independentes: um Torneio Eliminatório utilizando **Árvore Binária** e um Menu de Aplicativo utilizando **Árvore Genérica**.

O mapeamento das funcionalidades obrigatórias para os métodos da disciplina ocorreu da seguinte forma:

- **Modo 1 (Torneio):** A criação da chave foi feita de forma simétrica por recursão (divisão e conquista). Os métodos de percurso (Pré-ordem, Pós-ordem e Largura) foram adaptados para strings para exibir o andamento dos jogos. O cálculo do **LCA (Lowest Common Ancestor)** mapeou o exato ponto de cruzamento de dois jogadores na tabela (próximo confronto possível).
- **Modo 2 (Menu):** A inserção e remoção gerenciam uma lista dinâmica de filhos (List<Node>). A funcionalidade de **Mover Subárvore** foi mapeada alterando-se os ponteiros do nó-raiz da subárvore (limpando a referência do pai antigo e adicionando-a ao novo pai). O **Verificador de Consistência** mapeia um algoritmo de busca em largura utilizando uma fila de controle para interceptar ciclos de ponteiros.

2. Análise de Complexidade Big-O

Abaixo, justificamos a complexidade de tempo das principais operações implementadas:

Operação	Complexidade Temporal (Big-O)	Justificativa Teórica
Busca por texto (searchNodeRef)	$O(n)$ no pior caso	Como os nomes dos jogadores e opções de menu não seguem uma ordenação de árvore de busca (BST), é necessário fazer uma varredura completa (profundidade) visitando cada um dos n nós até encontrar a String correspondente.
Caminhamentos (Pré, Pós, Largura)	$O(n)$	Todos os algoritmos de percurso visitam exatamente uma única vez cada um dos n nós da árvore para imprimir ou coletar seus elementos.
Mover Subárvore (Menu)	$O(n)$ total (ou $O(1)$ após busca)	A alteração dos ponteiros do nó pai para o novo pai é uma operação de tempo constante $O(1)$. Porém, o processo exige encontrar o nó e o novo pai na árvore antes, o que eleva o custo

Operação	Complexidade Temporal (Big-O)	Justificativa Teórica
		total do método para $O(n)$ devido à busca.
Verificador de Consistência	$O(n)$	O algoritmo faz uma busca em largura utilizando uma fila auxiliar. Ele visita os nós um a um e faz uma checagem de contenção na lista de visitados. No pior caso (árvore sem ciclos), ele processa todos os n nós.
Cálculo da Altura / Folhas	$O(n)$	Requer uma passagem recursiva por toda a extensão da árvore para computar a profundidade máxima ou somar os nós terminais.